Protocolo de Aplicação

Serviço de troca de mensagens

Integrantes: Angelo Lemes (175744) e Arthur Lages (175757)

1. Visão Geral da Arquitetura e Comunicação

Arquitetura: Cliente-Servidor.

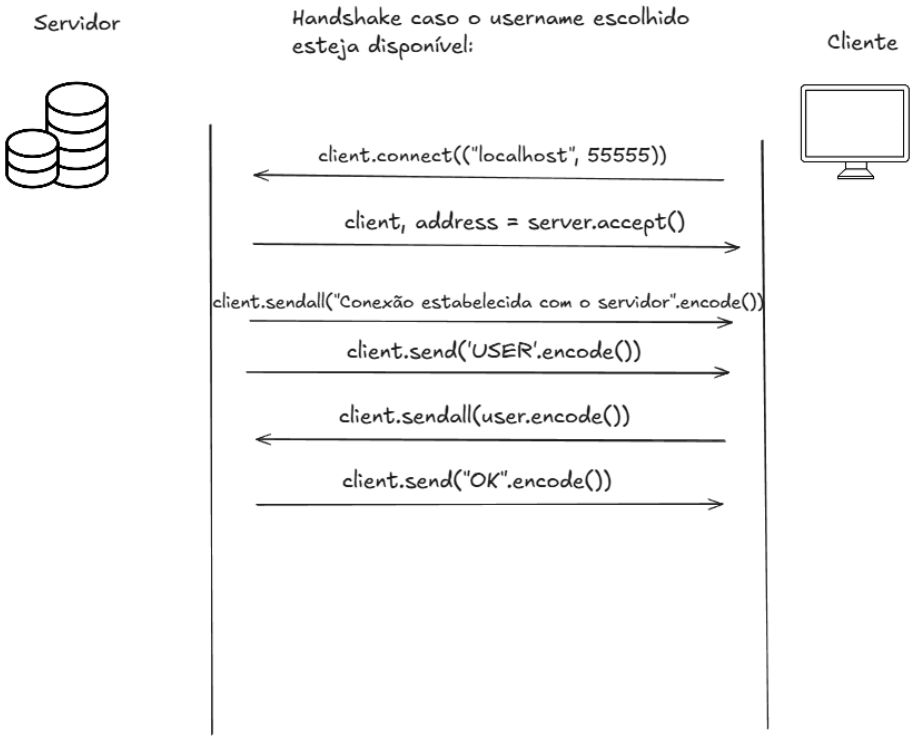
- Essa arquitetura foi escolhida pois o sistema precisa lidar com múltiplos participantes e grupos. Portanto, um servidor lidando com o gerenciamento de estados é a prática ideal, evitando que os clientes precisem se sincronizar uns com os outros. Além disso, os clientes não precisam ter conhecimento dos IPs uns dos outros. O cliente apenas aponta um nome de usuário, e o servidor encontra o IP do cliente correspondente àquele nome.

Protocolo de Transporte: TCP.

- O protocolo TCP foi escolhido pois, nessa aplicação, perder pacotes de dados ou receber segmentos fora de ordem poderia minar a coerência das mensagens. Além disso, o TCP sinaliza nativamente o encerramento de um socket, o que permite o fácil manejo do estado da aplicação.

Endereçamento: IPv4.
Codificação dos Dados: JSON codificado em UTF-8.

1.1 Handshake Inicial



2. Estrutura Padrão das Mensagens

Todas as mensagens trocadas no sistema utilizam um payload em formato JSON com os seguintes campos principais:

- **acao:** Cabeçalho para identificar o tipo de operação/mensagem recebida.
- **origem:** Identificador do cliente remetente.
- **destino:** Identificador do cliente ou grupo destinatário.
- **mensagem:** Conteúdo textual do payload enviado.

3. Especificação dos Protocolos de Comunicação

3.1 Mensagem Global (Broadcast)

```JSON
{  "acao": "BROADCAST",
  "origem": "cliente_origem",
  "mensagem": "conteudo_da_mensagem"}
```

Fluxo: Cliente -> Servidor -> Todos os clientes conectados.
Descrição: Transmite uma mensagem para todos os usuários conectados no momento.

3.2 Mensagem de Grupo

```JSON
{  "acao": "GROUP_MSG",
  "origem": "cliente_origem",
  "mensagem": "conteudo_da_mensagem"}
```

Fluxo: Cliente -> Servidor -> Membros do mesmo grupo.
Descrição: Envia mensagens para todos os membros que pertencem ao grupo do remetente.

3.3 Mensagens Privadas (Direct Message)

```JSON
{  "acao": "DIRECT",
  "origem": "cliente_A",
  "destino": "cliente_B",
  "mensagem": "conteudo_da_mensagem"}
```

Fluxo: Cliente A -> Servidor -> Cliente B.
Descrição: Envia uma mensagem direta e privada para um cliente específico.

3.4 Gerenciamento de Grupos

Convite para Grupo

```
JSON
{  "acao": "INVITE",
  "convidante": "cliente_origem",
  "convidado": "cliente_destino"}
```

Fluxo: Convidante -> Servidor -> Convidado.
Descrição: Envia um convite de entrada em grupo para um cliente específico.

Aceitar Convite de Grupo

```
JSON
{  "acao": "ACCEPT",
  "convidante": "cliente_convite",
  "convidado": "cliente_origem"}
```

Fluxo: Cliente -> Servidor.
Descrição: Confirma o aceite do convite e adiciona o usuário à lista de membros do grupo.

Sair do Grupo

```
JSON
{  "acao": "LEAVE_GROUP",
  "origem": "cliente_origem"}
```

Fluxo: Membro -> Servidor.
Descrição: Remove o usuário da lista de membros do seu grupo atual.

3.5 Mensagens do Servidor (Notificações/Alertas)

```
JSON
{  "acao": "SERVER",
  "mensagem": "conteudo_da_notificacao"}
```

Fluxo: Servidor -> Cliente(s).
Descrição: Notificações do sistema, alertas de erro ou recusas de comandos.

3.6 Atualização de Cache Local (Lista de Usuários)

```JSON
{  "acao": "USERS_LIST",
  "users": ["usuario1", "usuario2"]}
```

Fluxo: Servidor -> Todos os clientes.
Descrição: Atualiza a lista de usuários online mantida localmente por cada cliente.

4. Comandos da Interface do Cliente

- **/s [mensagem]:** Alterna para o canal geral ou envia uma mensagem global.
- **/p [mensagem]:** Alterna para o canal do grupo ou envia mensagem para o grupo (requer estar em um grupo).
- **/w <usuario> [mensagem]:** Envia mensagem privada para um usuário específico.
- **/inv <usuario>:** Convida um usuário conectado para formar/entrar em um grupo.
- **/accept:** Aceita o convite pendente para entrar em um grupo.
- **/leavep** ou **/lp:** Sai do grupo atual.
- **/online:** Exibe a lista de usuários conectados no momento.
- **/sair:** Encerra a conexão com o servidor de mensagens.
- /**help**: Mostra os comandos disponíveis.

5. Tratamento de Erros

5.1 Destinatários Inexistentes, Offline ou Inválidos

O próprio cliente, utilizando a lista local de usuários online, verifica se a mensagem está sendo enviada para si mesmo ou para um usuário inexistente/offline:

```Python
if alvo == user:
    print('ERRO: Você não pode mandar uma DM para si mesmo.')
```

```Python
if alvo not in lista_local:
    print(f'ERRO: O usuário mencionado {alvo} não foi encontrado ou está offline')
```

Descrição: Caso ocorra algum caso inválido, uma mensagem de erro é exibida e o usuário permanece no canal de comunicação atual.

5.2 Comandos Mal Formatados ou Desconhecidos

Quando um comando não segue a sintaxe esperada (como enviar o comando **/w** sem informar o destinatário), o cliente trata a entrada e exibe um erro de instrução:

```python
if len(partes) > 1:  # Testa se o usuário digitou um destinatário após o comando
	## Continua o protocolo de envio
	pass
else:  # Digitou apenas "/w" sem um destino. Uso incorreto
	print('Erro: Utilize a formatação /w <destino> [mensagem]')
```

```python
if entrada[0] == '/':
	partes = entrada.split(' ', 1) # Divide a entrada no primeiro espaço
	comando = partes[0].lower()

	if comando in ['/s', '/p']
		## Continuação do código.
	elif comando == '/w':
		## Continuação do código.
	elif comando in ['/invite', '/convidar', '/inv']: # convidou alguem pra um grupo
		## Continuação do código.
	elif comando in ['/accept', '/aceitar']:	# aceita convite de grupo
		## Continuação do código.
	elif comando in ['/leavep', '/leaveg', '/lp', '/lg', ]:	# sai do grupo
		## Continuação do código.
	elif comando == '/sair':  # sai do servidor
		## Continuação do código.
	elif comando == '/online': # mostra os usuários online
		## Continuação do código.
	elif comando == '/help':  # mostra a lista de comandos disponíveis
		## Continuação do código.
	else:
		# Usou "/" mas não digitou nenhum comando conhecido
		print('ERRO: Comando desconhecido.')
```

Descrição: Caso o comando digitado não exista, o cliente retorna a mensagem: "`ERRO: Comando desconhecido`".

5.3 Pacotes Corrompidos ou Formato Inválido

O servidor valida se a mensagem recebida é um payload JSON válido. Se a decodificação falhar, um alerta de erro é retornado ao cliente:

```python
try:
    message = json.loads(message) # tenta carregar json
except json.JSONDecodeError:
    server_msg('ERRO: Formato inválido ou pacotes foram corrompidos', [username])
    # não eh um json
```

5.4 Cliente tentando conectar caso o servidor estiver fechado

O cliente tenta conectar no servidor, e caso ele esteja fechado retorna o erro:

```python
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)      # define o cliente como (IPv4, TCP)
try:
    client.connect(("localhost", 55555))                        # conecta o cliente ao servidor
except:
    print('ERRO: Não foi possível conectar ao servidor.\nVerifique se o servidor está em execução e se o uso do endereço e das por
    exit(1)
```